

CALCITE TECHNOLOGIES

Internal Engineering Handbook

Stewardex

Subscription, Billing & Usage-Enforcement Implementation Handbook

A complete engineering specification for delivering tiered SaaS pricing, an admin pricing-control dashboard, and tenant-level usage metering and enforcement.

Field	Value
Document title	Stewardex Subscription, Billing & Usage-Enforcement Implementation Handbook
Audience	Project Manager, Backend Lead, Frontend Lead, QA Lead, DevOps
Prepared by	Sohaib Ahmed — Head of Engineering, Calcite Technologies
Prepared for	Stewardex Project Manager (internal delivery)
Companion document	Stewardex Pricing Strategy — Calcite Technologies (v1.0, 23 April 2026)
Document version	1.0
Date of issue	4 May 2026
Status	Approved for sprint planning
Classification	Confidential — Internal Engineering

Table of Contents

Table of Contents	2
1. Introduction	5
1.1 Purpose of this handbook.....	5
1.2 Scope.....	5
1.3 How to read this document.....	5
1.4 Stack reminder.....	5
2. Pricing Model Recap	7
2.1 Tier matrix.....	7
2.2 Module-by-module entitlements	7
2.3 The metered unit: workflow execution	9
2.4 Discount programs to model.....	9
3. Domain Model.....	11
3.1 Entity overview	11
3.2 Plan.....	11
3.3 PlanRevision (immutable history)	12
3.4 FeatureFlag	12
3.5 Subscription.....	12
3.6 SubscriptionOverride	12
4. Feature Flag Catalogue	14
5. Limits and Quotas.....	17
5.1 Limit types.....	17
5.2 Soft cap, hard cap, post-cap	17
5.3 First-overflow-free policy	18
5.4 Hard overflow cap (customer-set).....	18
6. Metered Event Catalogue.....	19
6.1 Event publication contract.....	20
6.2 Backfill rules for existing tenants	20
7. Roles and Access Control for Pricing	22
7.1 New and extended roles	22
7.2 The Calcite SuperAdmin domain	22
7.3 Two-person rule on dangerous actions	22
8. SuperAdmin Pricing Dashboard.....	24
8.1 Information architecture	24
8.2 Screen — Plans list (/admin/plans).....	24
8.3 Screen — Plan detail (/admin/plans/:code)	25

8.4 Screen — Tenant detail (/admin/tenants/:id).....	26
8.5 Screen — Coupons (/admin/coupons).....	27
8.6 Screen — Invoices (/admin/invoices).....	27
8.7 Screen — Audit (/admin/audit).....	27
8.8 Screen — Settings (/admin/settings)	28
9. Customer-Facing Billing Portal	29
9.1 Page layout	29
9.2 Plan change flow	29
9.3 In-product upgrade nudges.....	29
9.4 Soft cap and hard cap UX	29
10. Metering & Enforcement Engine.....	31
10.1 Architecture.....	31
10.2 Ingestion path.....	31
10.3 Aggregation path	31
10.4 Enforcement path (the fast path)	32
10.5 Stripe Billing Meter push.....	32
10.6 Live usage feed	32
10.7 Failure modes and what we do about them	32
11. Stripe Integration.....	34
11.1 Stripe object mapping.....	34
11.2 Webhooks	34
11.3 Stripe Customer Portal.....	34
11.4 Tax handling.....	34
11.5 Test mode discipline	35
12. API Surface.....	36
12.1 Admin endpoints (calcite.super_admin)	36
12.2 Tenant endpoints (tenant.org_owner / tenant.admin).....	36
12.3 System endpoints	37
13. Database Indexes and Performance	38
13.1 Caching	38
13.2 Sharding posture (forward-looking)	38
14. Security, Audit, Observability.....	39
14.1 Security	39
14.2 Audit	39
14.3 Observability	39
15. Testing Strategy.....	40

15.1 Test layers	40
15.2 Critical-path test scenarios	40
16. Phased Rollout Plan	42
16.1 Phase 1 — Foundations (Sprints 1–2)	42
16.2 Phase 2 — Metering core (Sprints 3–4)	42
16.3 Phase 3 — Stripe (Sprints 5–6)	42
16.4 Phase 4 — SuperAdmin Dashboard (Sprints 7–9)	42
16.5 Phase 5 — Customer billing portal (Sprints 10–11)	43
16.6 Phase 6 — Hardening (Sprint 12)	43
16.7 Cumulative timeline	43
17. Risks, Edge Cases, Open Questions	44
17.1 Risks	44
17.2 Edge cases worth calling out	44
17.3 Open questions for the PM and client	45
Appendix A — Quick Reference for Engineers	46
A.1 HTTP error contract for limit / feature gates	46
A.2 Acceptance test matrix — at-a-glance	46
A.3 Glossary	46
A.4 Closing note from Calcite	47

1. Introduction

1.1 Purpose of this handbook

This handbook is the authoritative engineering specification for implementing the Stewardex commercial pricing model, the admin-side controls that allow Calcite Technologies and the client (Mohammad) to operate that pricing without redeploying code, and the runtime metering layer that enforces tier limits in real time. It translates the commercial pricing strategy issued on 23 April 2026 into concrete engineering work — data models, API contracts, UI screens, background jobs, Stripe integration, and acceptance criteria — at a level of detail that the project manager can plan sprints from and the engineering team can pick up tickets from without further clarification.

Everything in this document is binding for delivery. Where the pricing strategy and this handbook conflict on a number, the pricing strategy is canonical for commercial intent and this handbook tells engineering how to make it true in the platform.

1.2 Scope

In scope: subscription plan modelling, plan particulars management (price, included quotas, feature flags, storage, seats), the SuperAdmin pricing dashboard, the customer-facing billing portal, Stripe Billing integration, the usage event ingestion pipeline, the metering and enforcement service, alerts and overage handling, the audit trail for all pricing actions, and acceptance tests.

Out of scope: anything that requires re-pricing the catalogue commercially (that is the pricing strategy document), tax-engine integration beyond Stripe Tax (deferred to a later release), and the full rebuild of the existing Stewardex modules (this handbook only adds the gating and metering hooks they need).

1.3 How to read this document

- Section 2 gives the recap of the pricing model so engineers do not have to keep two documents open.
- Sections 3–6 are the domain model: the entities, plans, limits, and what counts as a metered event.
- Sections 7–9 are the SuperAdmin pricing dashboard — what it must look like, what it must let an admin do, and what it must not let an admin do.
- Section 10 is the metering engine — the heart of the runtime enforcement story.
- Sections 11–13 are integrations (Stripe), API surface, and the customer billing portal.
- Sections 14–17 are non-functional concerns — security, audit, observability, testing, and the phased rollout plan with acceptance criteria the PM will use to sign off each sprint.

1.4 Stack reminder

Stewardex runs a React/Vite frontend on Vercel and a Node.js/Express backend on Render, with MongoDB Atlas as the primary store and AWS S3 for object storage. Stripe

(v17.3.1) is already integrated for one-off payment intents. The work in this handbook extends — it does not replace — that stack.

2. Pricing Model Recap

The commercial pricing model is a three-tier subscription with a metered overage on the workflow execution unit and a Community Impact discount for verified extra-small charities. Tiers, prices, included quotas, and feature gates are summarised below as the source of truth for engineering. Any divergence from this table at runtime — for example a customer being able to use a Professional-only feature on Foundation — is a P1 bug.

2.1 Tier matrix

Attribute	Foundation	Professional	Enterprise
Monthly list price (AUD)	\$299	\$1,299	\$4,999
Annual list price (AUD)	\$2,990	\$12,990	\$49,990
Annual save vs monthly	17%	17%	17%
Target customer	Small charities (\$50K–\$500K)	Medium charities (\$500K–\$3M)	Large charities (\$3M+) / groups
Staff seats included	5	25	Unlimited
Board seats included	9	Unlimited	Unlimited
Workflow executions / month	50	500	Unlimited (fair use)
Document storage	10 GB	100 GB	Unlimited (fair use)
Overage rate per workflow	AUD \$2.00	AUD \$0.50	—
Setup fee (monthly)	\$500	\$2,500	\$10,000
Setup fee (annual commit)	Waived	Waived	\$10,000 (not waived)
Support	Email, 2-BD	Priority, next-BD	Dedicated CSM, 4h P1
SLA	Best effort	Best effort	99.9% uptime
Trial	14 days, no credit card	14 days, no credit card	Guided POV with CSM

2.2 Module-by-module entitlements

The matrix below states which Stewardex modules are reachable on each tier. Reachable means: the customer can navigate to the module from the main menu and the module's API endpoints accept their request. Modules outside their tier must be visibly listed in the

navigation, but selecting them must redirect to an upgrade screen — never a 404 — to preserve the upsell surface.

Module	Foundation	Professional	Enterprise
Organisation & Responsible Persons register	✓	✓	✓
Board portal (agendas, packs, minutes, decisions)	✓	✓	✓
Policy Management (acknowledge, version control)	✓	✓	✓
Risk Register (standard matrix) & Incident register	✓	✓	✓
Complaints register	✓	✓	✓
Compliance Checklists (ACNC GS 1–6)	✓	✓	✓
AIS-to-ACNC submission workflow	✓	✓	✓
Immutable audit log	✓	✓	✓
MFA + role-based permissions	✓	✓	✓
Finance: expense, invoice, dual-approval	—	✓	✓
Budget entry & budget-vs-actual	—	✓	✓
Cash handling & donation-box tracking	—	✓	✓
Financial statements upload + board approval	—	✓	✓
Partner & Donor Vetting (KYC/AML/GDPR)	—	✓	✓
Meeting Minutes with e-signature	—	✓	✓
BAS / GST / PAYG quarterly workflow	—	✓	✓
Month-end & Year-end checklists	—	✓	✓
External Auditor scoped read-only access	—	✓	✓
AI Compliance Assistant (chatbot)	—	✓	✓
People / HR (induction, offboarding)	—	✓	✓
IT Register & Insurance tracking	—	✓	✓
Multi-entity / group management	—	—	✓
Custom workflows (no-code)	—	—	✓

Module	Foundation	Professional	Enterprise
Business Continuity Plan vault	—	—	✓
Secure Credential Vault	—	—	✓
Regulatory Radar	—	—	✓
SSO (SAML / OIDC) + SCIM	—	—	✓
REST API + webhooks	—	—	✓
White-labelling	—	—	✓
Data residency choice (AU primary / EU)	—	—	✓
SOC 2 / ISO 27001 evidence pack	—	—	✓

2.3 The metered unit: workflow execution

A workflow execution is a complete, audited business process resolving to a terminal state (approved, rejected, lodged, deposited, acknowledged-by-cohort, etc.). It is the atomic unit of value Stewardex delivers and the only thing that is metered for billing. Workflow executions are counted on the calendar month (UTC, then translated to the tenant's billing-cycle anchor) and reset to zero at the start of each cycle.

The exhaustive set of events that count as one workflow execution is defined in Section 6. Outside that list, no API call, page view, or background job is metered. This is a deliberately narrow definition — it keeps the meter honest and the customer's invoice predictable.

Engineering note: If a workflow has multiple steps (initiate, route to approver, approve, finalise) it counts as ONE execution at the moment it reaches a terminal state. Aborting before the terminal state does not count. This is how the customer expects it to work and how the invoice will be reconciled.

2.4 Discount programs to model

- **Annual commit** — ~17% off monthly equivalent. Encoded as the `annual_price` column on every plan; this is the price actually charged on annual renewal.
- **Two- and three-year commit** — 20% / 25% Year-2 / Year-3 discount applied as a Stripe coupon attached to the subscription, configurable from the admin dashboard.
- **Community Impact Program** — 60% off Foundation for the first year for verified extra-small charities. Modelled as a coupon plus a tenant flag; auto-expires after 12 billing cycles.
- **Volume / group discount** — 15% off list per entity for peak bodies / denominations purchasing for ≥5 sub-entities. Modelled as a parent organisation reference and a coupon on each child subscription.

- **Founding Customer** — 15% off list, price-locked for 24 months, available only in the first 12 months after launch. Modelled as a coupon with a hard expiry date and a flag preventing further price changes during the lock window.

3. Domain Model

This section defines the entities engineering must add to support tiered subscriptions and metering. Existing Stewardex entities (Tenant, User, Workflow, Document, etc.) are referenced where they are touched but are not redefined.

3.1 Entity overview

Entity	Purpose	Owner of writes
Plan	The catalogue definition of a tier (Foundation, Professional, Enterprise) and any custom plans created by SuperAdmin.	SuperAdmin only
PlanRevision	An immutable snapshot of a Plan's particulars at a point in time. Every change to a Plan creates a new PlanRevision.	System (on Plan write)
FeatureFlag	A named capability (e.g. finance.expense_workflow) that a Plan either grants or withholds.	SuperAdmin only
Subscription	A binding between a Tenant and a Plan, with the price actually charged, the billing cycle, and any tenant-specific overrides.	SuperAdmin + customer self-serve
SubscriptionOverride	A per-tenant deviation from the Plan default (e.g. give Acme Foundation 100 GB instead of 10 GB).	SuperAdmin only
UsageEvent	An immutable record that a metered action happened (workflow execution, storage delta, seat add).	Application code (write-once)
UsageCounter	A pre-aggregated snapshot of usage for a tenant for the current billing cycle, refreshed continuously.	Metering service only
Invoice	The Stripe invoice the tenant will pay for the cycle, including base subscription and overages.	Stripe webhook
BillingEvent	Audit trail entry for any pricing-relevant action (plan change, override, invoice issued, payment failed).	System (write-once)

3.2 Plan

The Plan entity is the single source of truth for a tier. It is the only place pricing or quota numbers live; nothing else in the codebase may hard-code a tier price, included seat count, included storage, included workflow count, or feature gate. If engineering finds such a literal anywhere else, it is a bug.

```
Plan { _id: ObjectId, code: String, // unique slug: 'foundation' | 'professional' | 'enterprise' | 'custom-<n>'
  name: String, // 'Foundation', 'Professional', 'Enterprise', etc. visibility: 'public' | 'private', // private = invite-only / custom
  status: 'active' | 'archived', pricing: { monthlyAUD: Number, // 299 annualAUD: Number, // 2990
    setupFeeMonthlyAUD: Number, // 500 setupFeeAnnualAUD: Number, // 0 (waived)
    overagePerWorkflowAUD: Number, // 2.00 | 0.50 | null currency: 'AUD', stripeProductId: String, // prod_...
```

```
stripeMonthlyPricelId: String, // price_... stripeAnnualPricelId: String, // price_... stripeOverageMeterId: String, //
meter_... Stripe metered price ID }, limits: { staffSeats: Number | -1, // -1 = unlimited boardSeats: Number | -1,
workflowsPerMonth: Number | -1, storageGB: Number | -1, apiCallsPerDay: Number | -1, // 0 = no API access
customWorkflows: Number | -1, childEntities: Number | -1, // multi-entity group }, features: { // map:
featureCode -> boolean 'finance.expense_workflow': true, 'finance.bas_lodgement': true, 'people.hr': true,
'auditor.read_only_access': true, 'ai.compliance_assistant': true, 'api.rest': false, 'sso.saml': false,
'group.multi_entity': false, // ... full list in Appendix A }, support: { channel: 'email' | 'priority' | 'dedicated-csm',
responseSLAHours: Number, uptimeSLAPct: Number | null, // null on tiers without contractual SLA }, trialDays:
Number, // 14 | 0 metadata: { description: String, targetCustomer: String, sortOrder: Number, //
controls display order on pricing page }, createdAt: Date, updatedAt: Date, updatedBy: ObjectId, //
SuperAdmin user }
```

Hard rule: Plan documents are write-restricted to SuperAdmin users (role = 'calcite.super_admin'). Tenant admins, board members, and ordinary staff cannot read this collection at all — they only see their resolved Subscription. The MongoDB role on Atlas must enforce this; do not rely solely on application-level checks.

3.3 PlanRevision (immutable history)

Every change to a Plan creates an exact snapshot in the PlanRevision collection — same shape as Plan plus the diff and the editor. PlanRevisions are append-only; nothing in the application is permitted to delete or update them. They serve three purposes: regulatory audit, billing dispute resolution, and the ability to render the pricing page exactly as it appeared on the day a customer subscribed.

```
PlanRevision { _id: ObjectId, planId: ObjectId, revisionNumber: Number, // monotonically increasing per
planId snapshot: { /* full Plan body */ }, diff: [ { path: 'pricing.monthlyAUD', from: 299, to: 319 } ], changedBy:
ObjectId, changedAt: Date, reason: String, // free-text required at write time }
```

3.4 FeatureFlag

FeatureFlag is a small lookup collection (think 30–60 entries) that defines every gateable capability in Stewardex. Each Plan grants a subset by setting features[code] = true. FeatureFlag definitions are managed by SuperAdmin only and cannot be deleted once a Plan references them — only deprecated.

```
FeatureFlag { _id: ObjectId, code: String, // e.g. 'finance.expense_workflow' category: String, //
'finance' | 'governance' | 'security' | 'integration' | 'ai' | ... name: String, // 'Expense workflow with dual
approval' description: String, // shown in admin UI status: 'active' | 'deprecated', createdAt: Date, }
```

3.5 Subscription

Subscription binds a Tenant to a PlanRevision (NOT to a live Plan). This is critical: when SuperAdmin changes a Plan, existing tenants stay on the revision they signed up under until SuperAdmin or the tenant performs an explicit migration. This protects customers from surprise price changes and gives Calcite a clean rollback path.

```
Subscription { _id: ObjectId, tenantId: ObjectId, planId: ObjectId, planRevisionId: ObjectId, // pinned revision
status: 'trialing' | 'active' | 'past_due' | 'canceled' | 'paused', billingCycle: 'monthly' | 'annual', cycleAnchor: Date,
// day-of-month this tenant rolls over currentPeriodStart: Date, currentPeriodEnd: Date, trialEndsAt: Date | null,
priceAUD: Number, // what they actually pay appliedCoupons: [ { code: 'COMMUNITY60', percentOff: 60,
expiresAt: Date } ], overrides: ObjectId | null, // SubscriptionOverride id, if any stripeCustomerId: String,
stripeSubscriptionId: String, paymentMethod: { type, last4, brand, expiresAt }, hardOverageCapAUD: Number | null,
cancelsAt: Date | null, createdAt: Date, updatedAt: Date, }
```

3.6 SubscriptionOverride

Per-tenant deviations from a Plan default. SuperAdmin uses this to negotiate one-off deals (e.g. give a particular Foundation customer 100 GB of storage instead of 10 GB

without creating a new Plan) and to grant Community Impact discounts. Overrides are clearly marked as such in the customer's billing portal so support conversations stay grounded.

```
SubscriptionOverride { _id: ObjectId, subscriptionId: ObjectId, limits: { storageGB?: Number, staffSeats?: Number,
workflowsPerMonth?: Number, ... }, features: { 'api.rest?': true, ... }, pricing: { monthlyAUD?: Number,
overagePerWorkflowAUD?: Number, ... }, effectiveFrom: Date, effectiveUntil: Date | null, reason: String,      //
required approvedBy: ObjectId,      // SuperAdmin user id createdAt: Date, }
```

Engineering note: When the metering service computes effective limits for a tenant, it merges `PlanRevision.limits` with `SubscriptionOverride.limits`, with the override winning field-by-field. Compute this once per request in middleware and stash it on `req.context.entitlements`; do not query the DB on every gate check.

4. Feature Flag Catalogue

Every gateable capability in Stewardex is named here and included or excluded per tier. The table below is the source of truth for the FeatureFlag collection and must be seeded into the database via migration on first deploy. New flags are added by SuperAdmin via the dashboard; flags cannot be deleted, only deprecated.

Code	Category	Description	F	P	E
governance.organisation	Governance	Org setup & Responsible Persons register	✓	✓	✓
governance.board_portal	Governance	Agendas, packs, minutes, decision register	✓	✓	✓
governance.policy_management	Governance	Policy lifecycle, acknowledge, version	✓	✓	✓
governance.risk_register	Governance	Risk register & matrix	✓	✓	✓
governance.complaints	Governance	Complaints register	✓	✓	✓
governance.incidents	Governance	Incidents register	✓	✓	✓
governance.compliance_checklist	Governance	ACNC GS 1–6 checklists	✓	✓	✓
governance.ais_workflow	Governance	Annual Information Statement workflow	✓	✓	✓
security.audit_log	Security	Immutable audit log	✓	✓	✓
security.mfa	Security	Multi-factor authentication	✓	✓	✓
security.rbac	Security	Role-based permissions	✓	✓	✓
finance.expense_workflow	Finance	Expense workflow with dual approval	—	✓	✓
finance.invoice_workflow	Finance	Invoice approval with compliance check	—	✓	✓
finance.budget	Finance	Budget entry & variance reporting	—	✓	✓
finance.cash_handling	Finance	Cash handling & donation-box tracking	—	✓	✓
finance.statements	Finance	Financial statements + board approval	—	✓	✓
finance.bas_lodgement	Finance	BAS / GST / PAYG quarterly workflow	—	✓	✓

Code	Category	Description	F	P	E
finance.month_end_checklist	Finance	Month-end & year-end checklists	—	✓	✓
finance.insurance_register	Finance	Insurance tracking integration	—	✓	✓
partner.kyc_aml	Partner	KYC / AML / GDPR partner & donor vetting	—	✓	✓
meeting.esignature	Governance	Meeting minutes with e-signature	—	✓	✓
auditor.read_only_access	Governance	External auditor scoped read-only role	—	✓	✓
ai.compliance_assistant	AI	OpenAI-backed AI compliance chatbot	—	✓	✓
people.hr	HR	Induction, offboarding, access revocation	—	✓	✓
it.register	Operations	IT subscriptions & web admin register	—	✓	✓
group.multi_entity	Enterprise	Parent / subsidiary multi-entity management	—	—	✓
workflow.custom_builder	Enterprise	No-code custom workflow builder	—	—	✓
governance.bcp_vault	Enterprise	Business Continuity Plan vault	—	—	✓
security.credential_vault	Enterprise	Secure Credential Vault	—	—	✓
governance.regulatory_radar	Enterprise	AU + intl regulator change monitoring	—	—	✓
sso.saml_oidc	Security	SAML / OIDC single sign-on	—	—	✓
sso.scim	Security	SCIM user provisioning	—	—	✓
api.rest	Integration	Public REST API	—	—	✓
api.webhooks	Integration	Outbound webhooks	—	—	✓
branding.white_label	Branding	Organisation branding on reports & portal	—	—	✓
data.residency_choice	Compliance	AU primary / EU on request	—	—	✓

Code	Category	Description	F	P	E
compliance.iso_soc2_pack	Compliance	Compliance evidence pack	—	—	✓
support.dedicated_csm	Support	Dedicated Customer Success Manager	—	—	✓
support.sla_99_9	Support	Contractual 99.9% uptime SLA	—	—	✓

Implementation note: Each gated module's controller must call `requireFeature('finance.expense_workflow')` at the top of the route; the middleware loads the tenant's effective entitlements from the request context and returns 402 `PAYMENT_REQUIRED` with an upgrade-CTA payload if the flag is false. Frontend interprets 402 by showing the upgrade modal; never a generic error.

5. Limits and Quotas

Limits are the second axis of plan particulars (the first being feature flags). Each limit type below has a defined unit, a defined reset cadence, and a defined enforcement behaviour at the soft cap, hard cap, and post-cap state.

5.1 Limit types

Limit	Unit	Reset cadence	Storage	Enforcement entry point
staffSeats	User accounts with role ≠ board	Never (point-in-time)	User collection count	Invitation send
boardSeats	User accounts with role = board	Never (point-in-time)	User collection count	Invitation send
workflowsPerMonth	Terminal-state workflow events	Monthly (cycle anchor)	UsageCounter	Workflow finalise endpoint
storageGB	Bytes in S3 + DB attachments	Never (point-in-time)	UsageCounter (continuous)	Pre-signed URL issuance
apiCallsPerDay	Authenticated REST calls	Daily (UTC midnight)	Redis sliding window	API gateway middleware
customWorkflows	Saved workflow definitions	Never (point-in-time)	WorkflowDefinition count	Workflow create endpoint
childEntities	Tenants under a parent group	Never (point-in-time)	Tenant.parentId count	Tenant create endpoint

5.2 Soft cap, hard cap, post-cap

Limit enforcement is three-stage. The same model applies to every limit, with the specific thresholds settable per limit per plan from the dashboard.

1. Soft cap (default 80% of included quota): the system fires an in-app banner and an email to the tenant admin warning that the limit is approaching. No action is blocked.
2. Hard cap (default = included quota; for unlimited limits, fair-use threshold defined per plan): two paths exist depending on the limit type.
 - For metered limits with overage pricing (workflowsPerMonth on Foundation/Professional): action proceeds, overage is recorded, customer is notified the next workflow will incur AUD \$2.00 / \$0.50.

- For non-metered limits (staffSeats, storageGB, apiCallsPerDay, customWorkflows, childEntities): action is blocked with HTTP 402 PAYMENT_REQUIRED and a structured upgrade-CTA payload.
- 3. Post-cap (only relevant where the customer set a hard overage cap of their own): subsequent metered events are still executed but billing is deferred to the next cycle for review.

5.3 First-overage-free policy

Every customer's first overage in their lifetime on a given subscription is automatically credited. This is set in code, not by SuperAdmin; the credit is recorded as a BillingEvent of type 'first_overage_credit' against the Subscription, with subsequent overages billed normally. This is an explicit churn-protection mechanism from the pricing strategy.

5.4 Hard overage cap (customer-set)

Customers can set an absolute monthly overage cap from their billing portal (e.g. 'never bill me more than AUD \$200 in overages this month'). When usage hits that cap, the metering service emits an event the application uses to: (a) continue accepting workflow finalisations because we will not refuse a compliance action over money, and (b) flag those subsequent workflows as 'pending review' on the upcoming invoice. The customer receives an email and an in-app banner.

6. Metered Event Catalogue

The events below are the complete list emitted by Stewardex application code via the `publishUsageEvent()` helper. The 'workflow execution?' column is what the meter increments. The 'storage delta' column says whether the event also adjusts storage usage. Anything not on this list is, by definition, free.

Module	Event name	Counts as workflow execution?	Increments storage?
Finance	expense.approved	Yes	Attachment delta
Finance	expense.rejected	Yes	Attachment delta
Finance	invoice.approved	Yes	Attachment delta
Finance	invoice.rejected	Yes	Attachment delta
Finance	payment.dual_approved	Yes	—
Finance	bas.lodged	Yes	Attachment delta
Finance	cash.deposit_confirmed	Yes	—
Finance	month_end.checklist_completed	Yes	—
Finance	year_end.checklist_completed	Yes	—
Governance	policy.acknowledgement_cycle_completed	Yes	—
Governance	policy.created	No	Document delta
Governance	board_pack.published	Yes	Attachment delta
Governance	meeting_minutes.signed	Yes	Attachment delta
Governance	decision.recorded	No	—
Governance	ais.lodged	Yes	Attachment delta
Governance	incident.closed	Yes	Attachment delta
Governance	complaint.closed	Yes	Attachment delta

Module	Event name	Counts as workflow execution?	Increments storage?
Governance	risk.review_completed	Yes	—
Partner	partner.kyc_completed	Yes	Attachment delta
Partner	donor.vetting_completed	Yes	—
People	induction.completed	Yes	Attachment delta
People	offboarding.completed	Yes	—
Custom	custom_workflow.terminal_state_reached	Yes	—
Compliance	checklist.gs_completed	Yes	—
AI Assistant	ai.chat_message_sent	No (rate-limited separately)	—
Audit log	audit.entry_written	No	—
File upload	document.uploaded	No	Document delta
File delete	document.deleted	No	Negative document delta
Auth	user.logged_in	No	—
Notification	email.sent	No	—

6.1 Event publication contract

Every metered event is emitted from the same helper, never inline in a controller. The helper is responsible for: stamping the tenant, the actor, the event timestamp, the workflow type, the request id (for traceability), and any storage delta, and writing it idempotently to the UsageEvent collection. Idempotency is keyed on (tenantId, eventId) where eventId is the workflow's terminal-state row id — duplicate publishes (from retries, double-clicks, etc.) collapse to one billed event.

```
// Backend pseudocode publishUsageEvent({ tenantId, actorUserId, eventCode: 'expense.approved', eventId: expense_id.toString(), // unique terminal-state id
countsAsWorkflow: true, storageDeltaBytes: attachmentSize,
occurredAt: new Date(), metadata: { expenseId, amount, approvers } });
```

6.2 Backfill rules for existing tenants

On the day the metering service goes live, no historical workflow execution counts toward the current cycle. The current cycle's counter starts at zero. Storage is, by contrast,

scanned and seeded once on activation — every existing document is sized and added to the storage counter — because storage is a point-in-time limit and a customer cannot suddenly have less of it than they had yesterday.

7. Roles and Access Control for Pricing

Pricing actions are higher-risk than ordinary tenant actions and need a tighter access control model. The roles below are introduced or extended in this work.

7.1 New and extended roles

Role	Where it lives	What it can do	What it cannot do
calcite.super_admin	Calcite-internal	Manage Plans, Plan particulars, Feature Flags, SubscriptionOverrides; view any tenant's billing; impersonate tenant admin for support.	Read tenant content (documents, minutes, financial data) — that is gated by separate per-tenant impersonation grant with audit trail.
calcite.billing_ops	Calcite-internal	View billing across tenants, issue manual credits, retry failed payments, attach coupons.	Edit Plan particulars or Feature Flags.
calcite.support	Calcite-internal	Read-only view of a tenant's subscription, usage, and invoice history.	Anything write.
tenant.org_owner (existing)	Tenant	View own subscription, change plan (with billing portal), set hard overage cap, download invoices.	Touch any other tenant's data; create Plans.
tenant.admin (existing)	Tenant	Same as Org Owner for billing.	Same restrictions.
tenant.staff / board (existing)	Tenant	See current plan name and an upgrade CTA when blocked by a feature gate.	View prices, invoices, or usage detail.

7.2 The Calcite SuperAdmin domain

All pricing administration happens on a separate web property, admin.stewardex.com (or a path-segregated `/calcite-admin` route on the same Vercel project — engineering choice, but the domain separation is the recommendation). It is reachable only from a hardened login flow with mandatory MFA, and access is restricted by IP allowlist for Calcite office and approved remote VPN ranges.

7.3 Two-person rule on dangerous actions

Three actions require a second SuperAdmin to approve before they take effect: (i) raising a Plan price, (ii) reducing any included quota on an active Plan, and (iii) deleting (deprecating) a FeatureFlag. The dashboard shows these as 'pending approval' until the

second admin clicks Approve; the action is logged as 'requested by X, approved by Y, executed at Z' in BillingEvent.

8. SuperAdmin Pricing Dashboard

This is the screen-by-screen functional specification for the Calcite-side dashboard that operates the pricing system. Every screen below has a dedicated route, an explicit set of inputs and outputs, and acceptance criteria the QA lead will use to sign off Sprint deliverables. Engineering may choose to colocate screens behind a single shell as long as the routes and behaviours are preserved.

8.1 Information architecture

Section	Route	Purpose
Home	/admin	At-a-glance: tenant count, MRR, overage revenue MTD, churn alert.
Plans	/admin/plans	List, filter, create, edit, archive Plans.
Plan detail	/admin/plans/:code	Edit a Plan's particulars; view its revision history; preview customer-facing pricing card.
Feature Flags	/admin/features	Catalogue of all gateable features.
Tenants	/admin/tenants	List, search, drill into a tenant's subscription.
Tenant detail	/admin/tenants/:id	Subscription, current usage, invoice history, override controls, impersonate-for-support.
Coupons	/admin/coupons	Manage coupons (Community Impact, Founding Customer, etc.).
Invoices	/admin/invoices	Cross-tenant invoice search, retry failed payments, manual credit.
Audit	/admin/audit	BillingEvent stream — every pricing-relevant action.
Settings	/admin/settings	Stripe keys, currency, tax mode, soft/hard cap defaults.

8.2 Screen — Plans list (/admin/plans)

Top-level table of Plans. Each row shows: code, display name, monthly AUD, annual AUD, included staff seats, included board seats, included workflows, storage, status (active / archived), revision number, and last-edited timestamp. The list is sortable on every column.

Primary actions

- Create new Plan — opens modal with the same particulars form as Plan detail but with all fields blank (or pre-filled from an existing plan if 'duplicate from' is chosen).

- Bulk archive — disabled when any selected Plan has active subscriptions on its current revision; when forced, a migration prompt is required.
- Export — CSV of the current plan catalogue for finance use.

8.3 Screen — Plan detail (/admin/plans/:code)

This is the most important screen in the entire admin dashboard. It is where the price, included quotas, and feature gates of a Plan are edited. Engineering must treat its acceptance tests as critical-path.

Layout

Single page divided into five tabs: (1) Pricing, (2) Limits & Quotas, (3) Features, (4) Support & SLA, (5) History.

Tab 1 — Pricing

- Monthly price (AUD) — number input, min 0, integer dollars and cents to 2dp.
- Annual price (AUD) — number input; UI shows the implied monthly equivalent and the implied discount % vs the monthly field, recomputed live.
- Setup fee monthly / annual — number inputs; setting annual to 0 shows 'waived' badge in the customer-facing preview.
- Overage rate per workflow (AUD) — number input, allows decimals; 'Disable overages' toggle sets to null and hides the line on the customer's invoice.
- Currency — read-only AUD for v1; field exists for future expansion.
- Stripe linkage — three read-only fields (productId, monthlyPriceId, annualPriceId, overageMeterId). A 'Sync to Stripe' button creates / updates the corresponding Stripe objects via the Stripe API; in v1 we never edit Stripe directly outside this dashboard.

Tab 2 — Limits & Quotas

- Staff seats included — number input, min 1; checkbox 'Unlimited' sets to -1.
- Board seats included — same.
- Workflow executions / month — same.
- Document storage (GB) — number input; 'Unlimited (fair use)' checkbox sets to -1 and exposes a 'fair-use threshold' field used for alerting only.
- API calls per day — number input; 0 means 'no API access' and is the v1 default for Foundation and Professional.
- Saved custom workflows — number input; only relevant when the workflow.custom_builder feature is enabled on the plan.
- Child entities — number input; only relevant when group.multi_entity is enabled.
- Soft cap % — number input, default 80, range 50–95.
- Hard cap % — number input, default 100; for tiers with overage, hard cap is informational; for non-metered limits it is the hard block.

Tab 3 — Features

Visual checklist of every FeatureFlag (grouped by category). Each row shows the flag code, name, description, and a toggle. A search box filters by code or name. Bulk actions: 'Apply

preset — Foundation defaults' / '— Professional defaults' / '— Enterprise defaults' load the canonical mappings.

Tab 4 — Support & SLA

- Support channel — single-choice: email / priority / dedicated-csm.
- Response SLA hours — number input.
- Uptime SLA % — number input or 'no contractual SLA' checkbox.
- Trial days — number input; 0 = no self-serve trial.

Tab 5 — History

Reverse-chronological list of PlanRevisions for this plan. Each row shows revision number, who edited, when, and a summary of the diff (e.g. 'monthlyAUD 299 → 319, workflowsPerMonth 50 → 75'). Clicking a revision opens a side panel with the full diff and a 'Roll back to this revision' button (which itself creates a new revision rather than overwriting history).

Save behaviour & two-person approval

Saving the form creates a new PlanRevision and updates the Plan document. If the save includes (a) raising any price, (b) reducing any included quota, or (c) removing any feature flag from an active Plan, the save is queued in a 'pending approval' state and a second SuperAdmin must approve from /admin/audit before the change takes effect. The originating admin sees a banner 'Awaiting approval from another SuperAdmin' and the changes do not propagate to Stripe or to live tenants until approved.

Migration of existing tenants

Plan particular changes never automatically migrate existing tenants — that is the whole point of pinning to PlanRevision. The save form has a final section 'Apply to existing tenants?' with three radio choices: (i) Do not migrate, (ii) Migrate at next renewal, (iii) Migrate immediately and prorate. Choosing (iii) requires a written reason and is itself a two-person approval action.

Acceptance criteria — Plan detail screen

4. Editing monthlyAUD from 299 to 319 and saving creates PlanRevision #N+1 with the diff captured.
5. Adding the api.rest feature to Foundation triggers the two-person approval banner because it is a quota change on an active tier; until approved, no tenant on Foundation can access the API.
6. Setting workflowsPerMonth to -1 on Professional displays 'Unlimited (fair use)' on the customer-facing preview and updates the Stripe metered price to be unbounded.
7. Setting overagePerWorkflowAUD to null hides the 'Workflow overage' line item on the next invoice generated for any tenant on this plan.
8. Saving fails closed with a clear error if Stripe returns a non-2xx on the Sync to Stripe step; PlanRevision is not written, application state is unchanged.
9. All saves write a BillingEvent with action 'plan.updated', the diff, and the actor.

8.4 Screen — Tenant detail (/admin/tenants/:id)

The single-tenant operational view used for support, custom-deal negotiation, and billing-dispute resolution.

Top section — Subscription summary

Current plan, plan revision pinned, billing cycle, current period start/end, trial status, payment method on file, hard overage cap, applied coupons, MRR contribution. A 'Change plan' button opens the migration flow. A 'Pause subscription' button pauses billing for up to 90 days (used in support cases where the customer is having a hard quarter).

Middle section — Current cycle usage

Live UsageCounter view: workflow executions used / quota, storage used / quota, staff seats used / quota, board seats used / quota, API calls today / quota. Each row shows a coloured bar (green / amber at soft cap / red at hard cap), the absolute number, and a 'reset' link that is disabled in production but available in staging for QA purposes.

Bottom section — Override controls

This is where SuperAdmin grants per-tenant deviations. The form is identical to the Plan detail Limits & Features tabs but every field is optional — only changed fields write into SubscriptionOverride. An 'Effective from' and 'Effective until' set the override window. A 'Reason' field is mandatory. Saving writes a SubscriptionOverride and a BillingEvent. The customer's billing portal renders the override clearly so support stays grounded ('You are on Foundation with 100 GB of storage on a custom arrangement until 31 Dec 2026.').

Side panel — Invoice history

Reverse-chronological list of Stripe invoices for the tenant. Each row links out to the Stripe invoice, with a 'Retry' button on past_due rows and an 'Issue credit (AUD)' button that creates a Stripe credit note via the API.

Side panel — Impersonate for support

Generates a one-time, time-boxed (15-minute), heavily-audited login link that lets a Calcite support engineer enter the tenant's UI as their org owner. Every action taken under impersonation is tagged with the impersonator's id in the audit log. The customer is emailed at the start of the session and shown a banner during it.

8.5 Screen — Coupons (/admin/coupons)

CRUD for Stripe coupons mirrored locally for searchability. Each coupon has: code, percent off OR amount off, max redemptions, redeem-by date, applies-to-plans list. Creating a coupon also creates the corresponding Stripe coupon via the API; archiving deactivates it on Stripe.

8.6 Screen — Invoices (/admin/invoices)

Cross-tenant invoice list with filters by status (paid / open / past_due / void), date range, and amount range. Bulk action: retry payment on all past_due invoices in one click. Manual credit issuance from this screen requires a written reason and is logged.

8.7 Screen — Audit (/admin/audit)

Full BillingEvent stream — every pricing action by every actor. Searchable by actor, action, target tenant, and date range. The 'pending approval' tab is where a second SuperAdmin reviews and approves or rejects two-person-rule actions. Approvals and rejections themselves write BillingEvents.

8.8 Screen — Settings (/admin/settings)

System-level settings, edited rarely. Includes: Stripe API mode (test / live, with prominent banner), default soft cap %, default hard cap %, default trial length, currency (read-only AUD for v1), and a 'Disable all overages globally' kill-switch used during incident response.

9. Customer-Facing Billing Portal

The portal lives in the existing Stewardex tenant app under `/settings/billing` and is visible only to users with role `tenant.org_owner` or `tenant.admin`. It is the customer's view of their plan, usage, and bills, and the place where they self-serve plan changes that do not require sales involvement.

9.1 Page layout

- Header — current plan name, billing cycle, next renewal date, payment method on file.
- Usage panel — same five rows as the SuperAdmin tenant view but without the staging-only reset link. Live, polled every 30 seconds via Server-Sent Events from the metering service.
- Plan options panel — 'You can upgrade to Professional' card with the per-tier diff highlighted; 'Switch to annual' suggestion if currently monthly; 'Cancel subscription' button that initiates the cancel flow with a retention prompt.
- Invoice history — last 24 invoices with download-PDF link to Stripe-hosted invoice.
- Hard overage cap — input field; saving sends the cap to Stripe and updates the local Subscription record.

9.2 Plan change flow

10. Customer clicks 'Upgrade to Professional'.
11. Modal shows a clear before/after: today AUD \$299/mo, from next cycle AUD \$1,299/mo. Annual option is offered alongside with the implied saving.
12. Customer confirms; if upgrading, the change is immediate and prorated; if downgrading, the change takes effect at the end of the current cycle (this protects revenue and prevents cycling abuse).
13. Stripe receives the subscription update. Webhook updates the local Subscription. A confirmation email is sent.

9.3 In-product upgrade nudges

When a tenant on Foundation tries to access a Professional-only feature (e.g. clicks Finance > Expense), they hit an upgrade modal — never a 404 or a generic permission error. The modal states the feature, the plan that includes it, the new monthly price, and a CTA to the Plan options panel. The modal copy is templated from a single source; do not allow individual modules to invent their own.

9.4 Soft cap and hard cap UX

- Soft cap — banner at the top of the workspace 'You have used 80% of your monthly workflow executions.' with a Dismiss action. Email also sent to org owner.
- Hard cap on metered limit — banner stays; subsequent workflows still execute and an overage line appears live in the Usage panel.

- Hard cap on non-metered limit — the action is blocked with a modal explaining why and offering an upgrade. The customer is not punished — the modal shows the exact additional cost so they can decide.

10. Metering & Enforcement Engine

The metering service is the runtime brain of this work. It records usage, aggregates it, exposes it for live UI, enforces limits at request-time, and feeds Stripe the metered counts that drive the invoice. This section is the engineering specification for that service.

10.1 Architecture

The engine is composed of four logical components. They can be deployed as a single Node service in v1 to minimise infrastructure overhead, with the option to peel off the ingestion path to a queue (BullMQ + Redis) if write volume justifies it later.

Component	Responsibility	Where it runs
Ingestion	Receive UsageEvent writes from application code; dedupe by (tenantId, eventId); persist.	In-process or BullMQ worker
Aggregation	Roll UsageEvents into UsageCounter snapshots; recompute soft/hard cap state.	Background worker, every 30s
Enforcement	Middleware that loads tenant entitlements & current counters and gates requests.	Express middleware in main API
Reporting	Push usage to Stripe Billing Meters; expose REST + SSE for UI.	Cron job (Stripe push) + HTTP endpoints

10.2 Ingestion path

Application code calls `publishUsageEvent()` at the moment a workflow reaches a terminal state, a document is uploaded or deleted, or a seat is added or removed. The helper writes to the `UsageEvent` collection with a unique compound index on (tenantId, eventId) so duplicate publishes (from retries, double-clicks, exactly-once-delivery failures) collapse to a single row.

Writes are append-only. No update, no delete. A nightly job archives events older than 24 months to cold storage in S3 (Glacier Deep Archive) and removes them from the hot Mongo collection. This is in line with the retention requirements of the audit log; nothing about it is changed by the metering work.

10.3 Aggregation path

A worker process runs every 30 seconds (configurable). For each tenant with new `UsageEvents` since the last run, it recomputes the `UsageCounter` document for the current billing cycle. The counter holds: `workflowExecutions`, `storageBytes`, `staffSeats`, `boardSeats`, `apiCallsToday`, `customWorkflows`, `childEntities`, plus `softCapHit` and `hardCapHit` booleans for each. The counter is the single piece of state the enforcement path reads — it is intentionally fast to query (one Mongo lookup keyed on `tenantId`).

```
UsageCounter { _id: ObjectId, tenantId: ObjectId, cycleStart: Date, cycleEnd: Date, workflowExecutions: Number,
workflowExecutionsByModule: { finance: Number, governance: Number, ... }, storageBytes: Number, staffSeats:
Number, boardSeats: Number, apiCallsToday: Number, customWorkflows: Number, childEntities: Number,
caps: { workflowExecutions: { soft: Boolean, hard: Boolean, overageWorkflows: Number }, storageBytes: { soft:
```

```
Boolean, hard: Boolean }, staffSeats: { soft: Boolean, hard: Boolean }, // ... }, lastEventAt: Date,
recomputedAt: Date, }
```

10.4 Enforcement path (the fast path)

The hot path on every authenticated tenant request is one Express middleware, `loadEntitlements`, that runs after auth and before any controller. It does the following, in order, with a 95th-percentile latency budget of 5 ms:

14. Look up the tenant's Subscription (cached in Redis with a 60-second TTL keyed on `tenantId`).
15. Resolve the effective entitlements by merging the pinned `PlanRevision.limits + PlanRevision.features` with any active `SubscriptionOverride`.
16. Look up the tenant's UsageCounter for the current cycle (cached the same way).
17. Attach `{ entitlements, counter }` to `req.context`.

Two helpers are then available to controllers and route definitions.

```
// Feature gate — used at the route level router.post('/expenses/approve',
requireFeature('finance.expense_workflow'), expenseController.approve); // Limit gate — used in the controller
before the action await requireLimitHeadroom(req, 'workflowExecutions', 1); // throws 402 if would exceed hard cap
on a non-metered limit await expenseService.approve(req.body); await publishUsageEvent({ ... countsAsWorkflow:
true ... });
```

Engineering note: `requireLimitHeadroom` takes a 'projected delta' so the controller can ask 'will adding ONE more workflow exceed the cap?' before running the action. For metered-with-overflow limits it never throws; it just returns metadata the controller may use to surface 'this will incur an overage' UX. For non-metered limits it throws a structured error the response middleware turns into 402 PAYMENT_REQUIRED with an upgrade-CTA payload.

10.5 Stripe Billing Meter push

A cron job runs every 15 minutes and, for each tenant on Foundation or Professional, pushes the delta in `workflowExecutions` since the last successful push to the corresponding Stripe Billing Meter. Stripe converts this into the overage line on the cycle's invoice automatically. We use Stripe's idempotency key set to `(tenantId, cycleStart, deltaCheckpoint)` so re-runs are safe.

On Enterprise, the metered price is configured but quantity is reported as 0 — Enterprise is included-unlimited, but the meter remains plumbed for visibility and for the eventual fair-use enforcement path.

10.6 Live usage feed

The customer's billing portal and the SuperAdmin tenant view both subscribe to a Server-Sent Events stream at `/api/usage/stream`. The metering service emits an SSE message every time the UsageCounter changes for a tenant the connection is authorised to view. Polling is also available for clients that cannot use SSE — same payload, 30-second cadence.

10.7 Failure modes and what we do about them

Failure	Effect	Mitigation
UsageEvent write fails	Risk under-counting overages.	Retry with exponential backoff; if it still fails, push to a dead-letter queue and alert ops.
Aggregation worker crashes	Counter goes stale; UI shows stale numbers but enforcement still works on last good counter.	Worker restarts on crash; a watchdog alerts if no run in > 2 minutes.
Stripe push fails	Invoice will under-bill; we recover automatically because deltas are idempotent.	Retries; alert on > 3 consecutive failures.
Redis cache miss storm	Latency spike to 30–80ms.	Hot Subscription documents are also cached in-process (LRU) for 5s as a defence in depth.
UsageEvent backlog from a tenant burst	Counter lags real usage by up to a minute.	Acceptable for billing (cycle is monthly) but bad for soft-cap UX. The aggregation worker scales horizontally and processes oldest-first.
Clock skew between API and metering worker	Cycle anchor confusion.	All timestamps stored in UTC; cycle anchors are computed from the Subscription record at boot.

11. Stripe Integration

Stewardex already runs Stripe v17.3.1 for one-off payment intents. This work extends the integration to cover full Subscriptions, Metered Billing, Coupons, and Customer Portal embedding.

11.1 Stripe object mapping

Stripe object	Stewardex equivalent	Where created
Customer	Tenant (1:1)	On tenant signup
Product	Plan	Sync to Stripe button on Plan detail
Price (recurring monthly)	Plan.pricing.stripeMonthlyPriceId	Sync to Stripe
Price (recurring annual)	Plan.pricing.stripeAnnualPriceId	Sync to Stripe
Price (metered)	Plan.pricing.stripeOverageMeterId	Sync to Stripe
Subscription	Subscription	On checkout success
Coupon	Coupon	Coupons screen
Invoice	Invoice (cached)	On Stripe webhook

11.2 Webhooks

Stripe is the system of record for billing state. Stewardex listens to the following webhooks and updates local state — never the other way around.

- `customer.subscription.created` / `.updated` / `.deleted` — mirrors into Subscription.
- `invoice.created` / `.finalized` / `.paid` / `.payment_failed` — mirrors into Invoice; `payment_failed` sends a banner to the tenant and a Slack alert to billing-ops.
- `customer.subscription.trial_will_end` — fires the 3-days-left email.
- `checkout.session.completed` — initial subscription activation.

11.3 Stripe Customer Portal

For payment-method updates and invoice downloads, embed Stripe's hosted Customer Portal rather than rebuilding it. The Stewardex portal links out to it via a one-time-use Customer Portal Session. This saves 4–6 sprint weeks of frontend work and removes us from PCI scope for card capture.

11.4 Tax handling

Australian GST is enabled via Stripe Tax. Tenant ABNs are validated on signup against the ABR and stored on the Stripe Customer object as a `tax_id`. Stripe handles the rate calculation, the inclusive/exclusive display, and the tax invoice generation. No bespoke tax logic is required in Stewardex code.

11.5 Test mode discipline

Calcite operates two Stripe accounts: live (production) and test (everything else). The `/admin/settings` page exposes a clear Test/Live banner. Migrations between modes are done by setting the `STRIPE_MODE` env var; no code changes. QA runs against test mode exclusively; production smoke tests against live mode use a single dedicated 'Calcite Internal QA' tenant with payments routed to a Calcite-owned card.

12. API Surface

This is the canonical list of new and modified endpoints. Authentication is the existing Stewardex JWT scheme. Authorisation is by role as defined in Section 7.

12.1 Admin endpoints (`calcite.super_admin`)

Method	Route	Purpose
GET	<code>/api/admin/plans</code>	List plans.
POST	<code>/api/admin/plans</code>	Create plan.
GET	<code>/api/admin/plans/:code</code>	Get plan + revisions.
PATCH	<code>/api/admin/plans/:code</code>	Update plan; creates a new PlanRevision; may queue for two-person approval.
POST	<code>/api/admin/plans/:code/sync-stripe</code>	Sync this plan's prices to Stripe.
POST	<code>/api/admin/plans/:code/archive</code>	Archive plan.
GET	<code>/api/admin/feature-flags</code>	List flags.
POST	<code>/api/admin/feature-flags</code>	Create flag.
PATCH	<code>/api/admin/feature-flags/:code</code>	Edit flag.
GET	<code>/api/admin/tenants</code>	Search tenants.
GET	<code>/api/admin/tenants:id</code>	Tenant detail with usage and invoices.
POST	<code>/api/admin/tenants:id/override</code>	Create / update SubscriptionOverride.
POST	<code>/api/admin/tenants:id/impersonate</code>	Generate one-time impersonation link.
GET	<code>/api/admin/coupons</code>	List coupons.
POST	<code>/api/admin/coupons</code>	Create coupon (also creates Stripe coupon).
GET	<code>/api/admin/audit</code>	BillingEvent stream.
POST	<code>/api/admin/audit:eventId/approve</code>	Two-person approval.
POST	<code>/api/admin/audit:eventId/reject</code>	Two-person rejection.

12.2 Tenant endpoints (`tenant.org_owner` / `tenant.admin`)

Method	Route	Purpose
GET	/api/billing/subscription	Resolved subscription + entitlements + usage counter.
POST	/api/billing/change-plan	Initiate plan change (proxies to Stripe).
POST	/api/billing/cancel	Cancel at end of period.
POST	/api/billing/hard-cap	Set or clear hard overage cap.
GET	/api/billing/invoices	Last 24 invoices.
GET	/api/billing/portal-session	Generate Stripe Customer Portal session.
GET	/api/usage/stream	SSE stream of UsageCounter changes for this tenant.

12.3 System endpoints

- POST /api/internal/usage/event — used only by application code; not externally callable. JWT-signed by the same backend service.
- POST /webhooks/stripe — Stripe webhook receiver; verifies signature; updates local state.

13. Database Indexes and Performance

The metering path is the only piece of this work that has meaningful performance constraints. The indexes below are mandatory.

Collection	Index	Purpose
UsageEvent	{ tenantId: 1, eventId: 1 } UNIQUE	Idempotent ingestion.
UsageEvent	{ tenantId: 1, occurredAt: -1 }	Tenant timeline queries.
UsageEvent	{ occurredAt: 1 }	Cold-storage archival job.
UsageCounter	{ tenantId: 1, cycleStart: -1 } UNIQUE	Hot-path counter lookup.
Subscription	{ tenantId: 1 } UNIQUE	Hot-path subscription lookup.
Subscription	{ stripeSubscriptionId: 1 } UNIQUE	Webhook reconciliation.
Plan	{ code: 1 } UNIQUE	Plan lookup.
PlanRevision	{ planId: 1, revisionNumber: -1 }	History view.
BillingEvent	{ tenantId: 1, createdAt: -1 }	Audit timeline.
BillingEvent	{ status: 1, createdAt: -1 }	Pending-approval queue.

13.1 Caching

- Subscription document — Redis 60s TTL keyed on tenantId, plus in-process LRU 5s TTL.
- PlanRevision — Redis 5min TTL (immutable, so safe).
- UsageCounter — Redis 30s TTL keyed on tenantId; written-through by the aggregation worker.
- Stripe Customer Portal sessions — never cached; one-time-use by definition.

13.2 Sharding posture (forward-looking)

MongoDB Atlas remains a single replica set in v1. If write volume on UsageEvent exceeds 5K writes/min sustained, shard on hashed(tenantId). UsageEvent is naturally tenant-keyed so no cross-shard queries are needed for the hot paths.

14. Security, Audit, Observability

14.1 Security

- All admin endpoints require role=calcite.super_admin (or a delegated role) and MFA-enforced session.
- Admin endpoints are also IP-restricted via Vercel + Render firewall rules to the Calcite office and approved VPN ranges.
- Two-person approval is enforced server-side, not just in the UI.
- Stripe API keys are stored in Render's secret manager; never in code, never in logs.
- Webhook endpoints verify Stripe signature; unsigned or mis-signed requests are rejected with HTTP 400 before any business logic runs.
- Tenant data isolation is preserved by the existing per-tenant database posture; the metering and pricing collections are exceptions and are global to Stewardex.

14.2 Audit

Every action on this list writes a BillingEvent record, append-only, ingestible by SIEM via the existing log pipeline.

- Plan create / update / archive.
- FeatureFlag create / update / deprecate.
- Subscription change (plan, cycle, status).
- SubscriptionOverride create / update.
- Coupon create / archive.
- Manual credit issued.
- Two-person approval requested / granted / rejected.
- Impersonation session started / ended.
- Hard-overage-cap changed (by tenant or by SuperAdmin).

14.3 Observability

- Metric: usage_events_ingested_total — counter, by tenantId, by eventCode.
- Metric: aggregation_lag_seconds — gauge, age of oldest unaggregated event per tenant.
- Metric: enforcement_middleware_latency_ms — histogram, p50/p95/p99.
- Metric: stripe_meter_push_failures_total — counter, alerted at > 3 consecutive.
- Metric: hard_cap_blocks_total — counter, by tenantId, by limit. (Used as an upgrade-conversion signal in Section 16.)
- Trace: every controller that calls publishUsageEvent annotates the active span with the eventCode.
- Log: every BillingEvent is also emitted as a structured JSON log at INFO.

15. Testing Strategy

15.1 Test layers

Layer	What it covers	Examples
Unit	Pure logic — limit math, entitlement merge, idempotency keying.	Merging PlanRevision + Override; computing soft-cap state; cycle-anchor maths.
Integration	Controllers + DB + cache + Stripe (mocked).	POST /api/admin/plans creates revision, syncs Stripe, returns 200.
End-to-end	Browser-driven against a seeded staging environment.	Foundation tenant tries to access Finance and sees the upgrade modal.
Contract	Stripe webhook payloads vs our handlers.	Replay every webhook in Stripe's library against staging and assert local state.
Load	Sustained ingestion under expected and 3× expected event rates.	Peak Ramadan donation-box scenario: 50 boxes × 5 events × 30 days ≈ 7,500 events / cycle, compressed into one day.
Chaos	Worker crash, Redis flap, Mongo step-down, Stripe outage.	Simulated outages must degrade UI, not corrupt billing.

15.2 Critical-path test scenarios

18. Foundation tenant exceeds 50 workflows: subsequent workflows execute, overage line appears on next invoice, first overage is credited automatically.
19. Foundation tenant exceeds 10 GB storage: next document upload returns 402 with upgrade modal; existing documents still readable.
20. Professional tenant added 26th staff seat: invitation rejected with 402 unless they upgrade or buy an additional seat add-on.
21. SuperAdmin sets storage override to 100 GB on Acme Foundation: Acme can upload until 100 GB; their billing portal shows 'On Foundation with 100 GB custom storage until 31 Dec 2026'.
22. Plan price raised from \$299 to \$319: existing Foundation tenants stay at \$299 (pinned to old revision); new signups get \$319.
23. Two-person approval: Admin A raises a price; the change is pending until Admin B approves; Stripe is not touched until approval.
24. Stripe webhook arrives twice (Stripe retries): handler is idempotent; local state changes once.
25. UsageEvent ingestion is called twice with same eventId: counter increments once.
26. Hard overage cap reached: subsequent workflows still execute; tenant emailed; events tagged 'pending review' on the upcoming invoice.

27. Tenant cancels mid-cycle: subscription stays active until period end; they retain access; renewal does not occur.

16. Phased Rollout Plan

This plan is sized for the existing two-FE / two-BE / one-QA team. Story points assume the team's current velocity (~20 SP / sprint per engineer). Sprint length is two weeks. The PM owns sequencing within phases; the dependencies between phases are non-negotiable.

16.1 Phase 1 — Foundations (Sprints 1–2)

- Domain models and migrations (Plan, PlanRevision, FeatureFlag, Subscription, SubscriptionOverride, UsageEvent, UsageCounter, BillingEvent).
- Seed data: three plans (Foundation, Professional, Enterprise) with the canonical particulars from Section 2.
- Feature flag catalogue seed.
- Entitlement-merge utility + unit tests.
- Indexes.
- Acceptance: API can resolve a tenant's effective entitlements in < 5 ms p95 against a populated test database.

16.2 Phase 2 — Metering core (Sprints 3–4)

- UsageEvent ingestion helper and idempotency.
- Aggregation worker and UsageCounter writes.
- loadEntitlements middleware, requireFeature, requireLimitHeadroom helpers.
- Audit logging via BillingEvent.
- Acceptance: a hand-written script can publish events at 1K/min and the counter stays < 60 s behind real time; gate helpers correctly block / pass / overage.

16.3 Phase 3 — Stripe (Sprints 5–6)

- Stripe Product / Price sync from Plan detail.
- Subscription creation flow with Stripe Checkout.
- Webhook receiver and idempotent local mirroring.
- Stripe Billing Meter push cron.
- Stripe Customer Portal embedding.
- Tax (GST) via Stripe Tax.
- Acceptance: a new tenant can sign up, choose a plan, pay with a Stripe test card, and the subscription appears in both Stripe and Stewardex with matching status.

16.4 Phase 4 — SuperAdmin Dashboard (Sprints 7–9)

- Admin shell, auth, IP allowlist, MFA enforcement.
- Plans list and Plan detail with all five tabs.
- Feature Flags screen.
- Tenants list and Tenant detail with override controls and impersonation.
- Coupons screen.
- Invoices screen.

- Audit screen with two-person approval workflow.
- Acceptance: SuperAdmin can perform every Section 8 action; the QA test pack from Section 15.2 passes.

16.5 Phase 5 — Customer billing portal (Sprints 10–11)

- /settings/billing page in the tenant app.
- Live usage panel with SSE.
- Plan change flow.
- Hard overage cap.
- Upgrade modal componentry and feature-gate integration into existing modules.
- Acceptance: a tenant org owner can self-serve every action listed in Section 9; QA scenarios from Section 15.2 pass; no module returns 404 in place of the upgrade modal.

16.6 Phase 6 — Hardening (Sprint 12)

- Load testing and chaos testing.
- Security review (internal); rotate any test credentials.
- Documentation refresh — runbooks, on-call playbooks, customer help-centre articles.
- Acceptance sign-off across PM, Engineering, and the client.

16.7 Cumulative timeline

End-to-end timeline assuming no contention with other Stewardex work: 12 sprints, ~24 calendar weeks. Aggressive but achievable parallelisation between Phases 4 and 5 can compress this by two sprints.

17. Risks, Edge Cases, Open Questions

17.1 Risks

Risk	Likelihood	Impact	Mitigation
UsageEvent under-counting due to retry storms	Low	High (revenue leak)	Idempotency key + dead-letter queue + nightly reconciliation report comparing UsageEvent count to module terminal-state counts.
Stripe outage during cycle close	Low	Medium	Push job retries with backoff for 24 h; on > 24 h escalate; Stripe issues invoices on its own clock so we can recover.
Plan particular change leaks to existing tenants accidentally	Low	High (customer trust)	Pinned PlanRevision + automated test that proves a Plan price change does not alter any active Subscription's price unless explicit migration is selected.
Soft-cap email storm	Medium	Low	One soft-cap email per tenant per cycle; in-app banner is the primary signal.
Two-person approval bottleneck when only one SuperAdmin is online	Medium	Medium	A second SuperAdmin from billing-ops can approve; weekend coverage rota agreed in advance.
Tenant disputes overage line	Medium	Low/Medium	BillingEvent + UsageEvent give every overage a complete provenance trail.
Migration of legacy tenants without subscriptions	High (one-off)	Medium	Migration plan in 17.3.

17.2 Edge cases worth calling out

- Tenant changes plan mid-cycle: Stripe handles proration; UsageCounter is rebased to the new plan's quotas at the moment of change; the soft/hard cap state is recomputed.
- Tenant downgrades: change takes effect at end of cycle; in-cycle usage is allowed up to the existing (higher) quota; no refunds of overage.
- Tenant cancels then reactivates: a new Subscription is created; usage history on the prior subscription is retained but does not roll forward.
- Workflow finalised in the last second of a cycle: counted in the cycle that contains its terminal-state timestamp, not the request timestamp.
- Storage delete: storage counter goes down; this never produces a credit because storage is a point-in-time limit.

- Seat removed: seat counter goes down; if a removal pushes the tenant from over-quota back into quota mid-cycle, no credit is issued.
- Multi-entity Enterprise group: each child entity has its own Subscription; billing rolls up to the parent for invoicing convenience but quotas remain per-entity.
- Tenant on the Community Impact Program graduates above \$50K revenue mid-year: program continues to its 12-month end; the tenant pays Foundation list price from month 13.

17.3 Open questions for the PM and client

28. Does Mohammad want a self-serve flow for the Community Impact Program (with verification by ABN look-up) or a Calcite-approved-only flow? Recommend Calcite-approved for v1.
29. Is white-labelling included in Enterprise base price or charged as the existing \$500/month add-on for Professional only? The pricing strategy treats it as included on Enterprise; engineering needs explicit confirmation before sprint planning.
30. Does the client want Stripe Tax invoice copies emailed to a finance distribution list automatically? Recommended yes; trivial to enable.
31. Annual price-lock for Founding Customers: do they renew at the locked price or step up to list at month 25? Recommend step-up with a 30-day notice.
32. Does the client want a CSV export of usage on a public API endpoint, or only via the billing portal UI? Recommend UI-only for v1.

Appendix A — Quick Reference for Engineers

A.1 HTTP error contract for limit / feature gates

```
HTTP/1.1 402 PAYMENT_REQUIRED Content-Type: application/json { "error": "feature_not_in_plan" | "limit_exceeded",
"feature": "finance.expense_workflow", // when feature_not_in_plan "limit": "workflowExecutions", // when
limit_exceeded "currentPlan": "foundation", "requiredPlan": "professional", "upgradeURL":
"/settings/billing#upgrade", "message": "Upgrade to Professional to use this feature." }
```

A.2 Acceptance test matrix — at-a-glance

Phase	Acceptance gate	Owner
1	Entitlement resolution < 5 ms p95	Backend Lead
2	Metering ingest 1K/min, lag < 60 s	Backend Lead
3	End-to-end Stripe signup against test mode	Backend Lead + QA
4	All Section 8 SuperAdmin scenarios pass	Frontend Lead + QA
5	All Section 9 tenant scenarios pass	Frontend Lead + QA
6	Load + chaos sign-off; security review pass	DevOps + QA

A.3 Glossary

Term	Meaning
Plan	Catalogue definition of a tier (Foundation, Professional, Enterprise, custom).
PlanRevision	Immutable historical snapshot of a plan.
Entitlement	The merge of plan limits / features with any tenant override — what the tenant can actually do right now.
UsageEvent	Append-only record of a metered action.
UsageCounter	Pre-aggregated cycle snapshot the hot path reads.
Soft cap	First threshold (default 80%) that triggers warning UX but does not block.
Hard cap	Threshold (default 100%) that blocks non-metered actions or starts overage on metered actions.
Overage	A metered action above the included quota, billed at the plan's per-event rate.
Two-person rule	A class of dangerous actions that require a second SuperAdmin to approve.

A.4 Closing note from Calcite

The intent of this handbook is operational clarity. If anything in here is ambiguous, that is a defect in the document — flag it back to the Head of Engineering and we will tighten it before it costs a sprint. Pricing is one of the few systems where a small mistake compounds into a hard refund conversation, so we have erred on the side of writing more, not less. Good luck — let us ship something the customer can trust on day one.

— End of document —